

Capstone Final Report

LLM-VC-Dungeon

Arad Fadaei, Mahboobeh Yasini, Johnpaul Tamburro

Version 0.2, April 9, 2026

Table of Contents

Abstract	1
1. Introduction	2
1.1 Problem Context	2
1.2 Project Objectives	2
1.3 Report Scope and Emphasis	2
2. System Overview and Methodological Framing	3
2.1 Architectural Principle: Deterministic Core with Generative Overlay	3
2.2 High-Level Runtime Flow	3
2.3 Current Repository Baseline	4
3. Milestone Progression Analysis	5
3.1 Milestone 01: Strategic Foundation (Contextual)	5
3.2 Milestone 02: Core Gameplay Loop and Hybrid Architecture (Primary Implementation Bridge)	5
3.2.1 Hybrid Command Model	5
3.2.2 Generation Refactor Strategy	5
3.2.3 State Synchronization Intent	6
3.2.4 Alignment with Current Codebase	6
3.3 Milestone 04: Runtime Hardening and MVP Consolidation (Primary Emphasis)	6
3.3.1 Deployment and Environment Standardization	6
3.3.2 Contract Expansion and API Hardening	6
3.3.3 Programmatic Gameplay Maturity	7
3.3.4 Generation and Narrative Reliability	7
3.3.5 Validation Expansion	7
4. Technical Implementation Synthesis	8
4.1 Backend Implementation Characteristics	8
4.2 Intent Interpretation Layer	8
4.3 Procedural Topology and Content Expansion	8
4.4 Frontend Interaction Surface	9
5. Evaluation, Reliability, and Engineering Quality	10
5.1 Evaluation Framing	10
5.2 Reliability Improvements Across Milestones	10
5.3 Practical Quality Indicators	10
6. Limitations and Validity Considerations	11
6.1 Current Limitations	11
6.2 Validity Considerations	11
7. Future Work	12
8. Conclusion	13

Abstract

This report presents the design, implementation trajectory, and technical consolidation of LLM-VC-Dungeon, a web-based interactive dungeon system with a terminal-styled frontend that combines deterministic game-state logic with large language model augmentation. The project objective was to reduce interaction friction found in traditional parser-based text adventures by accepting natural-language commands and translating them into structured gameplay actions. The implementation process evolved across milestone-defined phases: strategic architecture and risk framing (Milestone 01), hybrid gameplay loop implementation (Milestone 02), and runtime hardening with contract expansion and validation growth (Milestone 04).

The principal contribution is a hybrid architecture that separates deterministic mechanics from generative narration, thereby improving reliability without sacrificing narrative flexibility. Early planning artifacts included a voice-first direction, but voice I/O was later de-scoped and is not part of the supported runtime baseline. This report emphasizes Milestones, where the majority of implementation depth and engineering rigor are concentrated. Milestone 01 is treated as foundational context that informed subsequent architectural decisions. The resulting analysis characterizes design rationale, implementation outcomes, reliability improvements, and remaining limitations relevant to future development.

1. Introduction

1.1 Problem Context

Classic text adventures are effective at enforcing clear command grammar, but that strength also creates friction. Players often spend effort discovering acceptable syntax instead of engaging with the game world itself. In practice, this shifts attention away from narrative exploration and toward parser negotiation.

LLM-VC-Dungeon was developed in response to that usability gap. The project accepts natural-language input and maps it to structured gameplay actions through an LLM-assisted intent layer, while preserving deterministic state transitions in the backend. The core design challenge, therefore, is not only interpretive flexibility, but also the maintenance of reliable game logic under variable language input.

1.2 Project Objectives

The project pursued five linked objectives:

1. Replace rigid parser interaction with natural-language command interpretation.
2. Preserve deterministic game-state transitions through explicit action routing.
3. Support dynamic, theme-aware narrative generation.
4. Build a deployable web architecture with clear frontend-backend separation.
5. Improve runtime reliability through hardened contracts and validation pathways.

1.3 Report Scope and Emphasis

This report is grounded in implementation artifacts and milestone deliverables. Emphasis is assigned according to technical depth and impact on system maturity:

1. Milestone 04 (primary emphasis): runtime hardening, deployment standardization, and validation expansion.
2. Milestone 02 (secondary emphasis): hybrid command architecture and gameplay-loop maturation.
3. Milestone 01 (contextual emphasis): architecture strategy and scope definition.

2. System Overview and Methodological Framing

2.1 Architectural Principle: Deterministic Core with Generative Overlay

The primary architectural decision was to separate deterministic mechanics from generative narration. Deterministic logic governs action resolution, room traversal, inventory updates, and persistence. Generative components provide descriptive language and thematic variation.

This separation is central to the project's reliability strategy. If generative quality fluctuates, state correctness remains anchored in code-level rules. In other words, the model enriches the player experience but does not define game truth.

2.2 High-Level Runtime Flow

At runtime, each turn follows a consistent processing loop:

1. User input is collected from a web frontend with a terminal-styled interface.
2. Input is classified into an action schema by the intent layer.
3. The game engine applies deterministic state transitions.
4. The resulting state delta is converted into narrative output.
5. Updated state is persisted for continuity.

This flow is implemented through API orchestration in `app.main`, deterministic turn resolution in `game.game`, and model-backed intent and narration in `services.llm`.

The API-level turn entrypoint illustrates this orchestration boundary:

```
@app.post("/api/game/turn", response_model=GameTurnResponse)
async def process_game_turn(request: GameTurnRequest, background_tasks:
BackgroundTasks):
    """process a game turn with user input"""
    if not llm_provider:
        raise HTTPException(status_code=503, detail="llm provider not initialized")

    try:
        (narrative, action) = await game_engine.process_turn(request.user_input,
llm_provider, background_tasks)
        return GameTurnResponse(narrative=narrative, action=action)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"game turn failed: {str(e)}")
```

2.3 Current Repository Baseline

In the current repository baseline, the backend is implemented with FastAPI and exposes endpoints for health checks, game initialization, game loading, and turn processing. State is managed by a GameEngine that either restores prior sessions or initializes a new world, then persists updates after each turn. The Svelte frontend presents a terminal-styled web interface with command parsing, response streaming, and status feedback.

3. Milestone Progression Analysis

3.1 Milestone 01: Strategic Foundation (Contextual)

Milestone 01 established the initial project charter, deployment assumptions, and risk posture. Three outcomes were especially influential:

1. Scope narrowing to a multimodal web application, with desktop browser focus and explicit exclusion of high-complexity graphical targets.
2. Dual-stack architecture proposal (commercial cloud and fully self-hosted modes), allowing both deployment pragmatism and research-oriented flexibility.
3. Early evaluation strategy via synthetic command-to-action pairs to assess intent mapping quality.

Milestone 01 was intentionally strategic rather than implementation-heavy. Its value lies in architectural direction and scope discipline, both of which shaped later engineering decisions.

Not all initial assumptions persisted unchanged. In particular, the early voice-first direction was later removed. The supported baseline is now a web application with natural-language interaction through a terminal-styled frontend.

3.2 Milestone 02: Core Gameplay Loop and Hybrid Architecture (Primary Implementation Bridge)

Milestone 02 marked the project's transition from concept to playable architecture. The central contribution was a hybrid interaction model that reduced reliance on unconstrained model output for state-critical behavior.

3.2.1 Hybrid Command Model

The milestone formalized a two-lane command strategy:

1. System commands (for example, slash-prefixed interface controls) are handled deterministically and locally.
2. Narrative gameplay commands are routed through model-assisted intent interpretation.

This division improves responsiveness for control actions and avoids unnecessary model calls, while preserving expressive natural-language gameplay.

3.2.2 Generation Refactor Strategy

Generation was also restructured as a two-part process:

1. Programmatic structure generation (room graph, connectivity, baseline mechanics).
2. LLM-based narrative decoration (descriptions, thematic flavor, contextual prose).

Compared with full model-driven world creation, this design reduces token overhead and improves topological consistency.

3.2.3 State Synchronization Intent

A key Milestone 02 requirement was to expose player and room snapshots directly in API responses. This enabled reactive frontend updates while preserving the terminal-styled interaction model. Just as importantly, it laid the groundwork for the stricter interface contracts introduced in Milestone 04.

3.2.4 Alignment with Current Codebase

Current implementation elements that reflect Milestone 02 include:

1. Command/local handling scaffolding in the frontend terminal route.
2. Deterministic room topology generation and state transitions in backend game modules.
3. Tool-driven intent classification flow in the model service.

3.3 Milestone 04: Runtime Hardening and MVP Consolidation (Primary Emphasis)

Milestone 04 is the most technically mature phase of the project and represents its strongest engineering contribution. The focus shifted from feature introduction to operational hardening, observability, and reproducibility.

3.3.1 Deployment and Environment Standardization

Milestone 04 introduced containerized deployment artifacts for both frontend and backend, alongside compose orchestration and environment templates. This moved the project beyond ad hoc startup conventions and toward reproducible launch behavior suitable for demos and iterative validation.

3.3.2 Contract Expansion and API Hardening

The milestone also introduced concrete contract and safety upgrades:

1. Save/load path normalization with directory boundary enforcement.
2. Additional load and map endpoint capability for improved game lifecycle control.
3. Expanded turn-response snapshots exposing selected state metrics.
4. Diagnostic improvements in health payload visibility.

Collectively, these upgrades shifted the API surface from implicit behavior to explicit, inspectable contracts.

3.3.3 Programmatic Gameplay Maturity

Gameplay behavior was extended through deterministic mechanics, including:

1. Dedicated health action support.
2. Combat flow with damage and retaliation semantics.
3. Explicit game-over signaling behavior.
4. Better handling of revisit messaging and factual query persistence.

The practical effect was a reduced dependency on best-case model behavior for core progression.

3.3.4 Generation and Narrative Reliability

Generation reliability was reinforced through schema-aware retries, strict-mode safeguards, and canonical footer enforcement in narrative output. These controls reflect an important principle in production-facing LLM systems: expressive language is useful only when bounded by deterministic validation and fallback behavior.

3.3.5 Validation Expansion

Finally, Milestone 04 documented substantial test growth across map endpoints, turn handling, generation safeguards, and intent parsing. Even without a formal coverage target, this expansion signaled a transition from feature-centric iteration to reliability-focused engineering.

4. Technical Implementation Synthesis

4.1 Backend Implementation Characteristics

The backend uses FastAPI as an orchestration layer linking client input, intent classification, deterministic state updates, and narrative generation.

At the repository baseline:

1. Game initialization and load behavior are available through dedicated endpoints.
2. Turn processing applies intent classification, deterministic action execution, and narrative response generation.
3. State persistence is performed after each turn through JSON serialization pathways.

This arrangement supports predictable lifecycle control while preserving room for contract extension in future iterations.

4.2 Intent Interpretation Layer

The LLM service binds explicit action tools for intent classification and keeps classification distinct from narrative generation calls. This distinction is important: classification is constrained to a finite action vocabulary, while generation remains open-ended.

The action tool set used for constrained intent classification is explicitly enumerated in code:

```
ACTION_TOOLS = [
    action_move,
    action_look,
    action_take,
    action_attack,
    action_inventory,
    action_unknown,
]

class LLMService:
    def __init__(self):
        # bind action tools to classify subagent
        self.classify_agent = self.classify_agent_llm.bind_tools(ACTION_TOOLS)
```

Separating these concerns reduces ambiguity in control flow and keeps deterministic semantics in backend logic rather than in unconstrained model output.

4.3 Procedural Topology and Content Expansion

Dungeon topology is generated through graph-oriented expansion with bidirectional room links and a bounded room count. Content expansion combines structured prompting with JSON parsing,

output normalization, and fallback behavior when parsing fails.

In practice, this reflects three robust LLM-integration patterns:

1. Expect malformed outputs.
2. Normalize responses before parsing.
3. Apply fallback content to keep the loop non-blocking.

4.4 Frontend Interaction Surface

The frontend route implements a terminal-styled web interaction model with command registration, connection polling, and structured response logging. This design aligns with the project's low-friction, text-forward interaction goals while maintaining a clear path for iterative UX refinement.

5. Evaluation, Reliability, and Engineering Quality

5.1 Evaluation Framing

Given the capstone scope and the absence of external benchmark datasets, evaluation is framed around engineering reliability indicators rather than comparative model-scoring benchmarks.

The relevant evidence classes are:

1. API contract clarity and endpoint behavior consistency.
2. Deterministic game-state transition correctness.
3. Generation resilience under malformed or partial model output.
4. Reproducibility and deployability of runtime environment.

5.2 Reliability Improvements Across Milestones

The progression from Milestone 02 to Milestone 04 shows a clear reliability shift:

1. From feature availability to contract explicitness.
2. From ad hoc runtime assumptions to environment-standardized execution.
3. From implicit behavior to increasingly test-backed guarantees.

This trajectory is consistent with system maturation from prototype behavior toward MVP operational readiness.

5.3 Practical Quality Indicators

Several implementation signals support the project's quality claims:

1. Clear separation of concerns between interface, orchestration, state logic, and model services.
2. Deterministic fallbacks that preserve gameplay continuity under model uncertainty.
3. Structured endpoint design supporting lifecycle control and diagnostics.
4. Incremental move toward stronger validation discipline as reflected in Milestone 04 documentation.

6. Limitations and Validity Considerations

6.1 Current Limitations

The following constraints remain material:

1. Voice input/output is de-scoped and unsupported in the current project baseline.
2. Some frontend lifecycle commands are present as placeholders rather than complete gameplay actions.
3. Advanced balancing (damage curves, pacing, progression) remains an open tuning problem.
4. Strict runtime behavior in LLM-dependent modes remains sensitive to model availability, provider constraints, and configuration correctness.

6.2 Validity Considerations

Because the project prioritizes implementation maturity over formal user studies, its conclusions primarily support engineering feasibility and architectural validity rather than generalized player-experience outcomes. Future validation should therefore include controlled user testing, latency profiling across varied hardware and network conditions, and longer-horizon gameplay assessment.

7. Future Work

Future development should prioritize:

1. Completion of web command lifecycle features in production parity with backend capabilities.
2. Full implementation of save/load and map-centered command UX where currently scaffolded.
3. Extension of deterministic combat and progression systems with quantitative balance instrumentation.
4. Formalized test strategy publication (coverage targets, regression suites, and failure taxonomy).
5. Controlled evaluation of intent classification robustness against paraphrase, noise, and domain-specific vocabulary.

8. Conclusion

LLM-VC-Dungeon demonstrates that a deterministic-core, LLM-augmented architecture can reduce interaction rigidity in adventure-game interfaces while preserving state correctness. Milestone 01 established architectural direction and scope boundaries; Milestone 02 delivered the crucial hybrid interaction and generation bridge; Milestone 04 provided the strongest evidence of hardening through deployment standardization, contract expansion, and reliability-centered validation.

From a capstone standpoint, our project shows a credible progression from concept to an operational MVP baseline. The central engineering lesson is clear: practical LLM systems perform best when deterministic logic defines correctness and generative models operate within explicit, validated boundaries.